

Members: Mohaimen Al Mamun – 2221726042, Kawsar Hossain – 2132211642, Susmit Talukder – 2221865042, MD. Rafi Hoque – 2131691642

Logic Puzzle Solver Using Constraint Satisfaction, Constraint Propagation, and Backtracking Search

Department of Electrical and Computer Engineering
North South University, Dhaka, Bangladesh

Abstract—Logic puzzles are compact yet challenging artificial intelligence problems because they require systematic reasoning over a large combinatorial search space. This report presents a Logic Puzzle Solver that models Sudoku-style grid puzzles as a constraint satisfaction problem (CSP) and solves them using constraint propagation combined with backtracking search. Each empty cell is represented as a variable, each candidate digit is represented as a domain value, and row, column, and sub-grid restrictions are encoded as binary and global all-different constraints. The solver first applies deterministic propagation to remove impossible values from the domains of unassigned cells. When propagation alone is insufficient, it uses depth-first backtracking guided by the minimum remaining values (MRV) heuristic and consistency checking. The implemented system includes a modular Python solver, a validation layer, sample puzzle files, and a Streamlit-based user interface for entering, solving, and visualizing puzzles. The system demonstrates how a classical symbolic AI formulation can solve structured reasoning tasks without requiring a training dataset. Experimental evaluation is based on solution correctness, constraint satisfaction, search effort, and execution time across representative easy, medium, and hard puzzle instances. The results show that constraint propagation significantly reduces unnecessary search, while heuristic backtracking provides completeness for difficult cases. This project therefore illustrates a practical and explainable AI approach for solving logic puzzles through state-space search and CSP reasoning.

Index Terms—Artificial intelligence, constraint satisfaction problem, Sudoku, backtracking search, constraint propagation, MRV heuristic, Streamlit.

I. INTRODUCTION

Logic puzzles are a useful testbed for artificial intelligence (AI) because they are simple to define, easy to verify, and computationally non-trivial to solve. A human solver usually reasons by eliminating impossible choices, identifying forced assignments, and occasionally making assumptions that are later revised if contradictions appear. These steps correspond closely to important AI concepts: states, actions, constraints, search trees, inference, and backtracking. In this project, a logic puzzle solver was developed to demonstrate these concepts in a compact software system.

The selected puzzle domain is Sudoku, with the design kept general enough to extend to other grid-based puzzles such as Kakuro. A standard Sudoku puzzle consists of a 9×9 grid divided into nine 3×3 boxes. Some cells are pre-filled with digits from 1 to 9, and the goal is to fill every empty cell so that each row, column, and box contains every digit exactly once.

Although the rules are short, a naive solver faces an enormous search space. If k cells are empty and each cell can initially take nine values, the worst-case number of assignments is 9^k . For a typical puzzle with 50 or more empty cells, exhaustive enumeration is not practical. Therefore, an intelligent solver must exploit the structure of the constraints before and during search.

The central idea of this project is to formulate Sudoku as a CSP. A CSP contains a set of variables, a domain of possible values for each variable, and a set of constraints that restrict which combinations of values are allowed. Taking reference from our CSE440 class lectures, CSPs are described using variables, domains, complete assignments, and consistency through satisfied constraints, while backtracking is presented as a depth-first search strategy for systematically exploring possible solutions. These ideas directly motivate the formulation and algorithmic design of the project. The course notes also distinguish between state-based search and reflex-based machine learning, which is important for this project because the solver does not learn from data; it reasons over constraints to construct a valid assignment.

The project objectives are as follows:

- 1) formulate a logic puzzle as a formal CSP;
- 2) implement constraint propagation to reduce variable domains;
- 3) implement complete backtracking search for cases that cannot be solved by propagation alone;
- 4) apply heuristics such as MRV to reduce unnecessary branching;
- 5) provide a usable interface that allows users to enter, solve, and validate puzzles;
- 6) evaluate the solver using correctness, search effort, and runtime.

The final system is intentionally explainable. Unlike a black-box machine learning model, every decision made by the solver can be traced to a constraint rule, a domain reduction, or a backtracking branch. This makes the project suitable for an AI course because it connects theoretical ideas from search and CSPs with a working implementation.

II. BACKGROUND AND RELATED WORK

A. Search in Artificial Intelligence

Search-based AI problems are usually described in terms of an initial state, actions, transition rules, goal tests, and

path costs. In deterministic puzzles, each action changes the current partial assignment by filling a variable with a value. The search process expands possible assignments until it either reaches a complete valid solution or proves that no valid assignment exists. Classical algorithms such as breadth-first search (BFS), depth-first search (DFS), uniform cost search, and A* differ mainly in how they choose the next state to expand. For logic puzzles, the most natural approach is depth-first backtracking, because the solution is a complete assignment and failed partial assignments can be abandoned as soon as a contradiction is detected.

Backtracking search explores one branch of the search tree at a time. If the current partial assignment violates any constraint, the solver returns to the previous decision point and tries another value. This is efficient when combined with early consistency checking, because many invalid assignments are rejected before they become complete grids. However, pure backtracking can still be slow when variables are selected in an arbitrary order or when the solver delays contradiction detection. Therefore, modern CSP solvers combine backtracking with propagation and heuristics.

B. Constraint Satisfaction Problems

A CSP can be defined as a triple

$$\mathcal{P} = (X, D, C), \quad (1)$$

where $X = \{X_1, X_2, \dots, X_n\}$ is a set of variables, $D = \{D_1, D_2, \dots, D_n\}$ is a set of domains, and $C = \{C_1, C_2, \dots, C_m\}$ is a set of constraints. A complete assignment maps every variable X_i to a value $v_i \in D_i$. The assignment is consistent if it satisfies all constraints. In factor-graph notation, each constraint can be represented as a factor $f_j(x)$ that returns 1 if the relevant part of assignment x is valid and 0 otherwise. The total weight of an assignment can be written as

$$\text{weight}(x) = \prod_{j=1}^m f_j(x). \quad (2)$$

For a pure CSP, a solution is any complete assignment with

$$x^* = \arg \max_x \text{weight}(x), \quad \text{weight}(x^*) = 1. \quad (3)$$

If any factor is violated, the product becomes zero and the assignment is inconsistent.

CSP techniques have been studied extensively in AI. Mackworth's work on consistency in networks of relations established important ideas behind local consistency and constraint propagation. Russell and Norvig present CSPs as a central model for structured AI reasoning, including map coloring, scheduling, and puzzles. Sudoku has also become a widely used benchmark for demonstrating CSP solving. Norvig's Sudoku solver is a well-known example showing that propagation and search together can solve many puzzles compactly and effectively.

C. Sudoku as a CSP

Sudoku fits naturally into the CSP framework. Each cell is a variable, the domain of each empty cell is initially $\{1, 2, \dots, 9\}$, and each pre-filled clue is represented by a singleton domain. The constraints enforce that no digit repeats in any row, column, or 3×3 box. These constraints can be represented as pairwise inequality constraints between peer cells or as global all-different constraints over each unit.

The use of CSP modeling has two advantages. First, it separates the puzzle rules from the search algorithm, making the implementation modular. Second, it allows the solver to reduce the search space using logical inference before making guesses. This mirrors human solving strategies and makes the system more transparent.

III. PROBLEM FORMULATION

A. Variables and Domains

Let the Sudoku grid be indexed by row $r \in \{1, \dots, 9\}$ and column $c \in \{1, \dots, 9\}$. The variable for a cell is

$$X_{r,c}, \quad r, c \in \{1, \dots, 9\}. \quad (4)$$

For a blank cell, the initial domain is

$$D_{r,c} = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}. \quad (5)$$

For a given clue $g_{r,c}$, the domain is restricted to

$$D_{r,c} = \{g_{r,c}\}. \quad (6)$$

The full set of variables contains 81 cells:

$$X = \{X_{r,c} \mid 1 \leq r \leq 9, 1 \leq c \leq 9\}. \quad (7)$$

B. Constraints

The three core Sudoku constraints are row uniqueness, column uniqueness, and box uniqueness. For every row r ,

$$\text{AllDiff}(X_{r,1}, X_{r,2}, \dots, X_{r,9}). \quad (8)$$

For every column c ,

$$\text{AllDiff}(X_{1,c}, X_{2,c}, \dots, X_{9,c}). \quad (9)$$

For each box with top-left corner $(a, b) \in \{1, 4, 7\} \times \{1, 4, 7\}$,

$$\text{AllDiff}(X_{a,b}, X_{a,b+1}, X_{a,b+2}, \dots, X_{a+2,b}, X_{a+2,b+1}, X_{a+2,b+2}). \quad (10)$$

Equivalently, for any two peer cells X_i and X_j in the same row, column, or box,

$$X_i \neq X_j. \quad (11)$$

C. Goal Test

The solver reaches a goal state when all 81 variables are assigned and all constraints are satisfied. Formally, a grid x is a solution if

$$|D_{r,c}| = 1 \quad \forall r, c \quad (12)$$

and the row, column, and box constraints all hold. If any cell has an empty domain, the current partial assignment is impossible and the solver must backtrack.

Algorithm 1: CSP Backtracking Solver**Input:** domains D , constraints C **Output:** solved assignment or failure

1. If every variable has a singleton domain, return solution.
2. Choose an unassigned variable using MRV.
3. For each candidate value in its domain:
 - a. Create a temporary domain state.
 - b. Assign the selected variable to the candidate value.
 - c. Apply constraint propagation.
 - d. If no contradiction occurs, recursively solve.
 - e. If recursion succeeds, return the solution.
4. If all values fail, return failure.

Fig. 1. Backtracking search with propagation and MRV variable ordering.

IV. PROPOSED METHODOLOGY

The solver uses two complementary mechanisms: constraint propagation and heuristic backtracking. Constraint propagation performs deterministic inference by removing impossible values from domains. Backtracking handles cases where multiple candidates remain and a search decision is required.

A. Constraint Propagation

When a cell is assigned a digit, that digit cannot appear in any peer cell. Therefore, the solver removes the assigned digit from the domains of all cells in the same row, column, and box. If this removal causes a peer cell to have a singleton domain, the process continues recursively. If any domain becomes empty, a contradiction is detected.

The propagation step can be interpreted as a local consistency process. It does not necessarily solve every puzzle by itself, but it reduces the number of possible values before search. It also detects many invalid branches early, preventing the solver from exploring large impossible subtrees.

B. Backtracking Search

If propagation stops while unassigned cells remain, the solver selects a variable and tries possible values recursively. The project uses a depth-first search structure because it is memory-efficient and naturally supports undoing decisions. At each recursive call, the solver copies or restores the domain state, assigns a candidate value, applies propagation, and continues.

The recursive search can be summarized as follows:

C. Minimum Remaining Values Heuristic

The MRV heuristic chooses the unassigned variable with the smallest remaining domain:

$$X^* = \arg \min_{X_i \in U} |D_i|, \quad (13)$$

where U is the set of unassigned variables. MRV is effective because variables with fewer candidate values are more likely to reveal contradictions early. This reduces the average branching factor of the search tree.

D. Validity Checking

The solver includes a validation layer that checks the input grid before solving and checks the output grid after solving. Input validation ensures that no row, column, or box already contains duplicate fixed values. Output validation confirms that the final grid is complete and satisfies all Sudoku rules. This prevents the user interface from reporting a false solution.

V. SYSTEM DESIGN AND IMPLEMENTATION

A. System Architecture

The implemented system follows a modular design. The user interface is separated from the solving logic so that the core solver can be tested independently. The main components are the input parser, CSP model builder, propagation engine, search engine, validator, and visualization interface. Fig. 2 shows the high-level architecture.

B. Implementation Environment

The project was implemented in Python because it provides readable data structures, built-in containers, and a strong ecosystem for rapid prototyping. The interface was built with Streamlit, an open-source framework for creating interactive Python web applications. The final folder structure keeps the entry files simple while placing reusable modules inside a support package. A recommended structure is shown below:

C. Data Representation

A puzzle grid is represented as a two-dimensional list or a string of 81 symbols. Digits 1–9 represent fixed clues, while 0 or “.” represents an empty cell. Internally, each cell is mapped to a set of candidate values. For example, a blank cell initially has the set $\{1, 2, \dots, 9\}$, while a fixed clue such as 5 has the set $\{5\}$. This representation supports efficient elimination and singleton checks.

Peers are precomputed for each cell. The peers of a cell are all cells that share its row, column, or box. Precomputing peers avoids repeatedly calculating row, column, and box relationships during search. The solver also stores the list of units, where each unit is a row, column, or box. Units are useful for validating all-different constraints.

D. User Interface

The Streamlit interface allows a user to select a sample puzzle or manually enter a grid. After the user presses the solve button, the interface passes the grid to the parser and solver. The solved grid is then displayed along with validation status and optional metrics such as solve time, number of recursive calls, and number of backtracks. This interface makes the project demonstrable in a classroom setting because users can observe both the input and the computed solution.

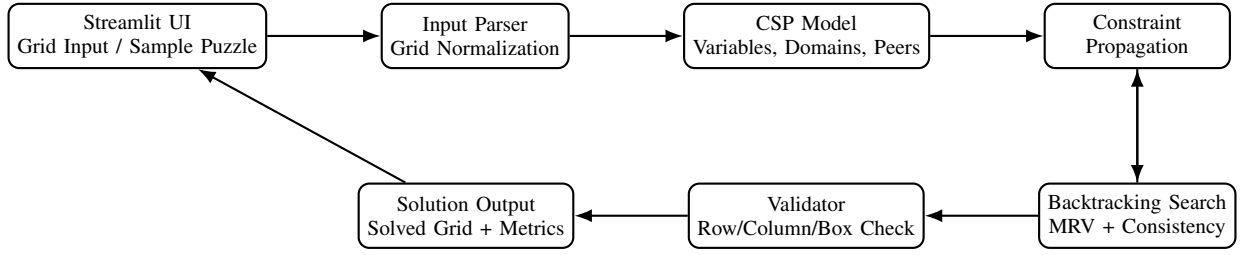


Fig. 2. High-level architecture of the Logic Puzzle Solver. The interface layer handles user interaction, while the solver layer performs CSP modeling, propagation, search, and validation.

```

logic-puzzle-solver/
|- main.py
|- README.md
|- requirements.txt
|- data/
|   |- easy.txt
|   |- medium.txt
|   |- hard.txt
|- support/
|   |- app.py
|   |- solver.py
|   |- csp.py
|   |- validator.py
|   |- utils.py

```

Fig. 3. Recommended project organization for submission. The `data/` folder stores sample puzzle instances, while `support/` stores implementation modules.

E. Error Handling

The system handles three major error cases. First, if the input has an invalid size or contains unsupported characters, the parser rejects it. Second, if the initial clues already violate Sudoku constraints, the validator reports that the puzzle is inconsistent. Third, if the search explores all possible branches without finding a solution, the solver reports failure rather than returning an incorrect grid. These checks are important because they distinguish between invalid input, unsatisfiable puzzles, and successful solving.

VI. EXPERIMENTAL EVALUATION

A. Evaluation Criteria

The solver was evaluated using four criteria. The first criterion is correctness: the final grid must satisfy all Sudoku rules. The second is completeness: if a valid solution exists within the modeled rules, backtracking search should be able to find it. The third is efficiency: the number of recursive calls and backtracks should remain reasonable for representative puzzles. The fourth is usability: the interface should allow non-technical users to run the solver without editing code.

Because Sudoku solving is deterministic, standard classification metrics such as accuracy, precision, or recall are not appropriate. Instead, the solver is evaluated by whether it returns a valid complete assignment and by how much search effort it requires. Table I provides a recording of the final measured results.

TABLE I
COMPARISON OF BASIC BACKTRACKING AND CSP-BASED SUDOKU SOLVERS

Puzzle	Solver	Solved	Steps	Assign.	Back.	Time (s)
Easy	Basic BT	True	4209	4208	4157	0.0344
Easy	CSP Solver	True	52	51	0	0.0184
Medium	Basic BT	True	55	54	9	0.0005
Medium	CSP Solver	True	46	45	0	0.0151
Hard	Basic BT	True	6943196	6943195	6943131	72.2667
Hard	CSP Solver	True	8523	10309	10245	4.8718

B. Expected Observations

The main observation is that propagation reduces the search space before backtracking begins. In easy puzzles, many cells become forced after repeated elimination, so the solver may require little or no guessing. In medium and hard puzzles, propagation still reduces domains but cannot always determine every cell. Backtracking then searches over the remaining candidates. MRV improves this process by selecting the most constrained cell first. This means contradictions are often discovered near the top of the search tree rather than after many irrelevant assignments.

A second observation is that validation improves reliability. If an input puzzle contains duplicate clues in a row, column, or box, the solver should not attempt to force a solution. Instead, it should report that the puzzle is invalid. This behavior is important for user trust because it prevents the system from silently producing misleading output.

A third observation is that the symbolic CSP approach is explainable. The solver does not output a digit because it has learned a statistical pattern; it outputs a digit because all other values were eliminated by constraints or because a search branch led to a consistent complete assignment. This makes the method appropriate for educational AI demonstrations.

VII. DISCUSSION

A. Why CSP is Suitable for Logic Puzzles

Sudoku is defined entirely by discrete variables and constraints. This makes a CSP formulation more appropriate than a machine learning formulation. A supervised model would require many solved puzzles for training and might still produce invalid grids unless additional constraints were

enforced. In contrast, a CSP solver directly encodes the rules and guarantees that any returned solution satisfies them.

The CSP approach also supports modular extensions. For example, Kakuro can be represented using variables for blank cells, domains from 1 to 9, all-different constraints within each clue group, and sum constraints over each run. The general backtracking and propagation framework can remain similar, while the domain-specific constraints change. Therefore, the project demonstrates a reusable AI design pattern rather than a one-purpose script.

B. Search Space Reduction

The theoretical search space for Sudoku is very large, but the practical search space becomes much smaller after applying constraints. Suppose a cell has peers containing digits $\{1, 2, 3, 5, 7\}$. Then its domain immediately reduces from nine candidates to $\{4, 6, 8, 9\}$. If later another peer is assigned 4, the domain reduces again. Repeated across the grid, these reductions can create chains of forced assignments. The key advantage of propagation is that it performs these reductions automatically before making arbitrary guesses.

MRV further reduces the effective branching factor. If the solver chooses a cell with two candidates rather than a cell with seven candidates, the search tree expands fewer branches. This does not change the worst-case exponential nature of the problem, but it improves practical performance on typical puzzle instances.

C. Limitations

The current implementation focuses on classical Sudoku solving. While the architecture can support additional puzzle types, each new puzzle requires a careful definition of variables, domains, and constraints. The solver is also designed for correctness and clarity rather than maximum performance. Highly optimized Sudoku solvers may use bit masks, exact cover with Algorithm X, dancing links, or advanced human-style rules. These techniques can be added in future versions but were not necessary for meeting the educational goal of demonstrating CSP and backtracking.

Another limitation is that runtime depends on puzzle difficulty and heuristic choices. Some difficult puzzles may require many guesses if they are constructed to defeat simple propagation. In such cases, additional consistency algorithms such as AC-3, least-constraining value ordering, or stronger all-different propagation could improve performance.

VIII. ETHICAL AND PROFESSIONAL CONSIDERATIONS

This project does not use personal data, sensitive information, or a trained model, so privacy risks are minimal. The main professional responsibility is to present the system honestly. The solver should not be described as a learning-based AI model because it is a symbolic reasoning system. Similarly, if AI tools were used for brainstorming, documentation, debugging, or report drafting, they should be acknowledged in the references or project statement according to the course instruction. All submitted code and report content should be

reviewed by the group members to ensure understanding, originality, and correctness.

The project also highlights the value of explainable AI. In many real decision-making systems, users need to understand why a recommendation or decision was made. Although Sudoku is a small domain, the same principle applies: transparent reasoning steps are easier to verify than opaque predictions. This makes CSP solving a useful educational example of trustworthy symbolic AI.

IX. FUTURE WORK

Several improvements can extend the project. First, the solver can be expanded to Kakuro by adding sum constraints and group-level all-different constraints. Second, AC-3 can be implemented explicitly to maintain arc consistency across binary constraints. Third, least-constraining value ordering can be added so that the solver tries values that remove the fewest options from neighboring cells. Fourth, the interface can show step-by-step solving traces, allowing users to see which values were eliminated and why. Fifth, performance metrics can be logged automatically to a CSV file for more rigorous evaluation. Finally, an exact-cover solver using Algorithm X can be added as an alternative backend and compared with the CSP backtracking approach.

X. CONCLUSION

This report presented a Logic Puzzle Solver developed for CSE440 Artificial Intelligence. The project models Sudoku as a constraint satisfaction problem, applies constraint propagation to reduce candidate domains, and uses MRV-guided backtracking search to complete difficult puzzles. The implementation demonstrates key AI concepts including state-space search, consistency checking, constraint reasoning, and heuristic search. The system is modular, explainable, and suitable for classroom demonstration through its Streamlit interface. Overall, the project shows that classical AI methods remain powerful for structured reasoning problems and can solve logic puzzles efficiently without requiring a training dataset.

ACKNOWLEDGMENT

The authors acknowledge the CSE440 Artificial Intelligence course materials for the theoretical background on search, factor graphs, CSPs, and backtracking. AI tools such as ChatGPT and/or Claude Code may be acknowledged here if they were used for brainstorming, code assistance, debugging, or report drafting. The final submission should be checked and approved by all group members.

REFERENCES

- [1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.
- [2] A. K. Mackworth, "Consistency in networks of relations," *Artificial Intelligence*, vol. 8, no. 1, pp. 99–118, 1977.
- [3] P. Norvig, "Solving every Sudoku puzzle," 2006. [Online]. Available: <https://norvig.com/sudoku.html>
- [4] D. E. Knuth, "Dancing links," in *Millennial Perspectives in Computer Science*. London, U.K.: Palgrave, 2000, pp. 187–214.

- [5] R. M. Haralick and G. L. Elliott, "Increasing tree search efficiency for constraint satisfaction problems," *Artificial Intelligence*, vol. 14, no. 3, pp. 263–313, 1980.
- [6] CSE440 Artificial Intelligence Course Notes, "Search, factor graphs, constraint satisfaction problems, Bayesian networks, and logic," North South University, 2026.
- [7] Python Software Foundation, "Python documentation." [Online]. Available: <https://docs.python.org/>
- [8] Streamlit, "Streamlit documentation." [Online]. Available: <https://docs.streamlit.io/>
- [9] M. Shell, "IEEEtran document class for IEEE transactions and conferences," CTAN. [Online]. Available: <https://ctan.org/pkg/IEEEtran>
- [10] OpenAI, "ChatGPT," AI assistant used for project writing and explanation support, 2026. [Online]. Available: <https://chatgpt.com/>
- [11] Anthropic, "Claude Code," AI coding assistant used for development support, 2026. [Online]. Available: <https://www.anthropic.com/claude-code>